MQ Platform

Supporting documentation v 1.0

Table of Context

Glossary 3

Overview 4

Games and Modes 5

- Max Quest games 7
  - SP Mode 7
  - BTG Mode 10
- Crash games 11
  - SP Mode 11
  - BTG Mode 13
- Game History 14

Tools 15

Integration 17

Server Side 19

- GS 21
  - Basic entities 22
- MQ 23
- Build and Deploy 24
  - Build 24
  - Deploy 24

Client Side 24

Glossary

Bank — one of the main entities of the MQ Platform. Contains information about the integration of one online casino, including endpoints for the necessary operations, available currencies, games, integration configurations, and more.

Buy-In — fixed bet for 1 round in BTG game mode. The amount of bets from all players per round forms the pot, which goes to the winner.

BSG Platform — platform from «BetSoftGaming» that allows to implement and integrate slot and action casino games.

EC (External Client) —company/person who integrates MQ Platform games.

Environment_domain — The environment domain provided by MQ Platform side. Used for requests from the EC System to MQ Platform services/games. For example, to launch games.

GS_domain — domain for the GS server application allowing access to any resources of this server. For example, to tools that allow to change any configuration.

MQ_domain — domain for the MQ server application allowing access to any resources of this server. For example, to tools that allow to change any configuration.

MQS — series of multiplayer online casino games in the «Shooter» genre.

Subcasino — uniting all banks of one licensee/EC entity.

Overview

MQ Platform is a platform that allows to implement web action casino games and provides the ability to integrate these games into casino sites.

Action casino game is a multiplayer game that allow players to fully interact with the game world using real money and have the opportunity to win real money. However, the payout is determined only by the mathematical models of the games. MQ action games also have several modes allowing to play either against the computer or against other players.

The platform is a client-server application that implements Action Casino games and provides the ability to configure the integration of these games on the EC side (casino web sites). The server part of the application consists of 2 applications:

- **Game Server (GS) App** is the main application responsible for performing such processes as interacting with the database, launching games, integrating with the casino (EC), and more.
- **Max Quest (MQ) App** is an application that implements the main logic of action games, game rooms, game math and more.

MQ application can only work together with GS application. Communication between applications takes place using Thrift.

The server part of the platform must be deployed on at least 2 instances of the cluster. Each instance must be running peer application instances. The minimum number of instances for a cluster is determined by the possibility of normal application updates without interrupting game activity, as well as greater fault tolerance. More detailed cluster requirements are described in the «Build and Deploy» section.

The client part of the application implements the display of the game to the user, as well as the interaction of the player with the game, and is implemented mainly using HTML5 and JS framework «Pixi.JS». Also, some client-side modules are coded in Javascript without the use of third-party frameworks. More detailed information about the technologies used in the client part in the section «Client Side».

The interaction between the game client and the server takes place via a WebSocket connection.

MQ Action games have certain browser requirements, which are described in the «Games and Modes» section.

The MQ Platform allows to integrate games into casino sites, however, the player's balance is on the side of the casino (EC). Integration allows MQ Platform to perform such HTTP requests as authorization of the player, withdrawal of a bet from the player's balance, credited winnings to the player's balance. For more information about integration, see the "Integration" section.

Games and Modes

MQ Platform currently implements 2 types of Action Casino games:

- **Max Quest** is a series of multiplayer shooter casino games. The player can fire shots at various enemies on the map. Each shot is a player's bet. With each shot, the player can get a win. The winnings are calculated according to the mathematical model of the game.
- **Crash games** are multiplayer casino games in which a player places a bet on a certain lifetime of an object during a round. The end of the round is the self-destruction of the object. The longer the player's bet existed, the more he has a win.

Both series of games have different modes:

- **SP** — player's game in a common room with other players, but against the casino. That is, the classic mode, where the casino pays the player in case of winning.
- **BTG** — game mode in a common room with other players, but against these players. All players during the game receive points, not real money winnings. To start the round, all players make a fixed amount - a bet (buy-in). The bets of all players per round form the pot of this room. The player with the most points wins the pot minus the casino commission. The casino makes money on commissions.
- **CAF** — BTG in private rooms. Similar to matchmaking.

Each game has its own unique identifier used for launch and all subsequent entries. The SP game mode has a different gameId from the BTG game mode, as well as a different launching process. CAF game mode has the same gameId as BTG mode but has a different launching process. More details about the launch in the section «Integration».

Each game can have its own configurations, which are tied to the bank and currency. Also, the game can have template settings for all systems, which can be override. Some bank settings may also affect the game.

Each game can have localizations for different languages. The language in which the game will be launched is indicated in the link launching the game according to the integration protocol. All current games on the MQ platform have a single localization – «en», with the exception of Triple Cash or Crash (863).

Both series of games in any mode are multiplayer and use rooms to implement the multiplayer mode. However, in the SP versions of the games, the player can be alone in the room, as the SP game always goes against the casino. At the same time, at least 2 players in the room are required to play BTG mode.

Each game has a round system. That is, one or more players make bets in one round and receive a win in this round (or at the end of it).

Each launch of the game by the player creates a game session. Each game session may have one or more game rounds. In some exceptional cases, a game session may be empty (contain no game rounds), but these are exceptions to the normal behavior of the system.

Also, since the game is multiplayer and has a system of rooms in which several players play at once, the games have such a parameter as RoomRoundID.

That is: for example, 3 players got into the same room and played one round there. RoomRoundID could have a value like "455673". At the same time, each of the 3 players for this round will have their own game session with a unique ID and also their own unique game round ID.

At the moment, the following games are implemented on the MQ platform and are available for integration:

Game Name	Mode(-s)	GameId
Max Quest: Mission Amazon	SP	859
Max Quest: Mission Amazon	BTG, CAF	862
Max Quest: Dragonstone	SP	838
Max Quest: Dragonstone	BTG, CAF	856
Max Quest: Sector X	SP	866
Max Quest: Sector X	BTG	867
Triple Max Blast	SP	875
Max Blast Champs	BTG	864
Triple Cash or Crash	SP	863

All these games except «Triple Cash or Crash» are exclusive games that must be integrated only by the sole licensee (EC) - Max Quest Brand. «Triple Cash or Crash» is already being integrated by various licensees and will not be integrated by this licensee.

Implementation of the CAF mode requires improvement of the BTG version of the game. For games «Max Quest: Sector X» and «Max Blast Champs» CAF mode is not implemented at this stage.

Max Quest games

The basic idea of games in this series is to shoot at certain moving targets (monsters, targets, and, in the specifications, enemies) with different weapons, where each shot has a certain cost, and each hit gives a chance for a certain payout. The payout and the probability of that payout for a hit are determined by the game's metamathematical model for each target and weapon individually.

During the game round, the player is playing in a «room», sometimes with other players in SP mode and always in BTG/CAF mode. The capacity of the room is determined at the design stage. Each player has several variants of weapons from which he can shoot, and which come in both standard and special (which have a higher cost per shot and can give a higher gain on a hit).

Also, in addition to different weapons and targets, there are two modes/versions for MQ games — BTG and SP.

- **SP-mode or Standard Play** — players play against the casino, which means that all winning payouts are paid by the casino. Each shot is a bet, and hitting can give a chance for a certain payout.
- **BTG or Battleground mode** — mode that does not change the game mechanics of the SP version, however, players compete for the pot of the round, and the casino earns on commission from each pot. That is, the players play against each other. The player with the most points wins the pot, minus the casino commission (rake). The bet is a fixed amount before the start of the round (buy-in). All shots are shareware, special weapons can only drop as a bonus during the game (cannot be bought).

Both versions of the games have a balancing act for the number of players in the rooms to eliminate the possibility of empty rooms.

There is also a **CAF mode** — private rooms for the BTG game mode. That is, the player invites other players to the BTG mode with whom he wants to play. At the same time, only invited players can enter the room.

SP Mode

The SP mode of the game assumes the existence 1 or several players in the game room, but each player plays against the casino (against the game). That is, multiplayer in this mode is only to raise the interest of players. Each of the players gets the winnings only for the enemies that he destroys.

The following main points describing MQ games can be distinguished:

- Betting
- Rooms and rounds
- Shooting and weapon mechanics
- Goals and trajectories
- Boss mode

Betting

The bet in the MQ games is every shot. Each shot has its own cost. There can be several bet levels in one room (like «bet per line» in slot casino games).

Before entering the game, the player must choose the base cost of a shot (bet) from the available ones. For the player, this is represented by a betting interval spanning all levels of that base bet. There can be several players in the same room, but only with the same base bet. That is, players with the same bet can only play in the same room, but by changing the level of this bet, each shot can have a different cost.

The bets available to the players are the configurable amount of the game.

To ensure continuous shooting, a certain amount of money is debited from the player's balance. Those, a certain amount is reserved from which the cost of each shot is deducted during the game. As soon as the reserved amount ends, the next reserve of funds is debited from the player's balance.

If at the end of the round the player has money left in the reserved amount, then they will be returned using the credit operation. For correct statistics on bets and winnings, the integration protocol implements additional parameters that allow to distinguish between the reservation of funds from the real bet, as well as the real winnings from the returned funds. More details in the section «Integration».

Rooms and rounds

After a player has chosen base bet to play, the system places him in a room with the players who have chosen the same base bet.

A room is a playful space, which can contain one or more players with the same selected betting range. At the moment in the implemented MQ games, there can be up to 6 players in one room. Before the start of each round, the player sees a timer that counts down the time until the start.

As soon as the start timer expires and there are enough players in the room, a limited time game round is started. If the maximum number of players in the room is reached before the timer ends, the round starts immediately regardless of the timer.

After the end of the round, the player receives all winning funds, as well as a refund of unspent reserved funds. The system then begins counting down to a new round.

Shooting and weapon mechanics

After starting the game, the player has the ability to fire shots from standard weapons (turret) and special weapons. A shot from a standard weapon with level 1 is equal to the base bet of the room. The player can choose different levels of standard weapons, depending on this, the cost of a shot will change as the base bet multiplied by the level multiplier of the selected turret.

The cost of one shot from a special weapon is determined as the cost of a shot from the player's chosen bet and bet level (turret) multiplied by N, where N is an integer multiplier depending on the selected special weapon. That is, a shot from a special weapon is equal to several shots from the selected turret.

The higher the cost of the shot, the greater the win the player can get, however, all wins are determined by the mathematical model of the game.

There are also shooting modes in the game: single shot by pressing LMB, an automatic gunfire sequence by pressing and holding LMB and Auto-fire mode, where player can choose a target by clicking on them with RMB.

Enemies and trajectories

All enemies appear and move on the round map according to preset trajectories (see game specifications). Appearance, type, and additional mechanics are also predetermined.

Shots that hit a target may or may not result as a win (calculated by a mathematical model), regardless of whether the target was destroyed or not.

Enemies are also divided into «normal», «elite» or «rare» (the description of the classing of the targets depends on each game). «Normal» targets appear more often and give a small win, while «elite/rare» targets may have additional mechanics and give an increased win (in relation to normal targets). Some of the «elite» targets may show a special animation after being shot.

Also, most MQ games have so-called «formations» of targets. In other words, targets appear on the map as a predetermined group, where each target has its own starting point and the path of movement (either same trajectory for all (temporal formation) or defined for each target separately).

Boss mode

In addition to «normal» and «elite/rare» targets, the game may also have a Boss fight mode. In this case, a different-looking target appears on the screen, and the boss's HP scale appears on the screen, which will decrease when the shots are successfully hit the boss.

Shooting the boss can, with a certain probability, give or not give a win (calculated by the mathematical model), which is also calculated separately for each shot at the boss. In the specifications this is called «partial win». If the boss is killed, the system gives the player who hit the boss last time an extra win to what he already got.

Trajectories on the map, as well as appearance and special mechanics are determined for each boss individually.

BTG Mode

This version of the game takes the existing mechanics and features from the SP version of the game and adds interaction between players — instead of playing against the casino, players play against each other, earning points by destroying targets. The one with the most points take the winnings (pot of the round).

In other words, the BTG version differs from the SP version in that:

1. The player pays the Buy-in to enter the round at the start of the game.
2. Only players with the same Buy-in can be in the same room.
3. With each player's Buy-in a Pot is formed - the total pot of winnings for a particular round. During the round the Pot is shown as a whole, i.e. the commission is not taken into account.
4. From each Buy-in the system collects Rake — a commission for the «use of the game». Rake is charged only if the round is successful.
5. All winnings from monsters are converted into points, which are then added to the player's score. All players in the room form a leaderboard from the player with most points to least.
6. The player with the most points by the end of the round becomes the winner and takes all the winnings, i.e. the entire Pot (minus rake) is credited to the balance of the winner.
7. If there are several winners (i.e. several players have equal amount of points and all come on the first place), then the system divides the winnings between winners equally.
8. Buy-in must be entered/confirmed for each round.
9. All shots from any weapon are free. SW and turret levels can only be obtained by destroying the target. By default, at the beginning of the round the player has only a level 1 turret available.
10. The BTG round is shorter in time than the round in the SP version of the game.
11. After killing a boss all players' scores from the current boss fight are doubled.

Crash games

Crash games is a series of multiplayer casino games where the main idea is to bet on the existence of an object for a certain amount of time. The object on which the bet is placed should exist only for a certain amount of time at the end of which it should self-destruct. The closer a player's bet is to the time the object is destroyed, the larger the payout he can receive.

Players do not know when the object is destroyed, so the duration of the round varies greatly from a few seconds to a few minutes. depending on the maximum multiplier of the rocket (set when configuring the game on the bank). That said, if the object crashes before the time, on which the player placed a bet, or if the player bet at the same time the object was destroyed, he loses the bet.

The game also has the concept of «rooms» in which there can be several players.

In the current implemented Crash games, the object of existence is a rocket. Each bet is an astronaut who is placed in this rocket. Each round begins with a rocket taking off. During the flight of the rocket, the multiplier increases, which determines the winnings. To get a win, the player must eject the astronaut from the rocket before the explosion. The win is equal to the bet multiplied by the multiplier at the time the astronaut was thrown out. If the player did not eject the astronaut before the rocket exploded, then he loses his bet.

Like MQ games, the Crash game can also be designed in several variants/versions:

- **SP mode or Standard play** — mode, in which players in the same room play against the casino. Each player has the ability to place the bet he wants, as well as win by ejecting his character («astronaut») from the rocket at any time from the start to the crash of the rocket.
- **BTG or Battleground mode** — just like the MQ games, it is also based on the SP version, adding interaction between players. In this mode, each player pays to enter the round (Buy-In), from which the bank of the round (Pot) is then formed. The player who ejects his astronaut from the rocket last becomes the winner and takes the entire pot of the round.

Both versions of the games have a balancing act for the number of players in the rooms to eliminate the possibility of empty rooms.

SP Mode

Rooms and rounds

Like other MQ games, Crash is based on a room architecture, but in this case the player is placed in a completely random room when the game starts, where a round is about to begin. Only players with the same currency can be in the same room.

The player selects the bet on a panel via special field and options of incresing/decreasing bet.

The capacity of rooms in Crash-games is much larger than in MQ — it can reach up to 30 players in one room.

Once the player starts the game and enters the room, he also sees a timer before the start of the round. During this timer the player has an opportunity to place a bet (the number of bets depends on the versions of the game, in the current versions it is a maximum of 3 bets from one player per round). If the timer expired, the round starts even if the player was alone in the room and did not have time to place a bet - the rocket flies anyway, the multiplier increases. In other words, if the player did make a bet, he takes part in the game round and can eject his astronaut.

At a certain point (calculated by the server) the rocket crashes and the round ends. All players, whose bets were withdrawn before the explosion (i.e. their astronauts ejected out of the rocket), receive their winnings into their balance in accordance with these bets. Players whose bets remained in the rocket and were not taken out (i.e. their astronauts were not ejected) lose their bets.

Rocket and the process of calculating winnings

During the round the rocket starts moving on a part of the playing field, and along with this the winnings multiplier starts increasing. The higher the rocket flies, the higher goes the multiplier.

A player can eject his astronaut during a rocket flight and receive winnings equal to his bet, multiplied by the multiplier at which he ejected his astronaut. Also the player's nickname is highlighted in the player's table accordingly.

The rocket can crash at any time during a round (calculated by the server part, but players do not see this), so the duration of each round is very different. As soon as the rocket crashes, all players remaining in it lose their bets.

Auto-Eject

An additional feature available between rounds that allows players to set a multiplier at which the system ejects the player's astronaut automatically, thereby «playing the bet» for him. This bet can also be lost if the rocket explodes before reaching the designated multiplier. In current versions of the games it is called Auto-cashout.

BTG Mode

The basic mechanics of the BTG game are still that players must eject their astronauts before the rocket crashes, but this time only the last ejected astronaut wins.

The winnings are not multiplied in any way, because they are made up of the player's Buy-in's, forming the Pot. The multiplier in this case is replaced by the «distance of flight».

In other words, the BTG game differs from the SP version in that:

1. The multiplier is replaced by the flight distance and no longer increases the winnings.
2. A player can only have one astronaut, i.e. only one bet per round (buy-in is only one per round).
3. Players pay the Buy-in at the start of the game.
4. Only players with the same Buy-in can be in the same room.
5. With each player's Buy-in a Pot is formed — the total pot of winnings for a particular round. I.e. if 5 people entered the room and each of them put 1000$, the total Pot will be 5000$.
6. From each Buy-in the system collects Rake - a commission for the «use of the game». The commission is charged only if the round is successful.
7. All players in the room form a leaderboard, where the last ejected is on the 1st place and the others are sorted in descending order. For each player the system shows how much time and distance his astronaut flew.
8. At the end of the round the player who ejected his passenger last, i.e., has the longest flight time and distance, becomes the winner, and takes all the winnings. I.e., the whole Pot (minus rake) is credited to the player-winner's balance.
9. If there are several winners (i.e., several players ejected his passengers at the same moment and their distance and time values coincide), then the system divides the winnings between the winners equally.
10. Buy-in must be entered/confirmed for each round.
11. There is no Auto-cashout feature (at the time of writing this feature has been postponed for the future).

Game History

VBA (Visual Bet Archive) is a functionality for viewing the game session by rounds, the history of a particular player session. I.e. through it the user can see what bet was made in each round, what was the winnings, what monster he shot (if it is MQ games).

A game session is a set of rounds in a particular game of one player. A game session consists of rounds. There can be several bets and wins in one round.

The VBA is stored for the last three months. Therefore, if a game session ended more than three months ago, an error will be displayed when trying to view the VBA of a round for such a session.

VBAs are stored on the server side (GS) in Cassandra. VBAs can be «retrieved» from it by gameSession_id and round_id.

Tools

A set of special tools is used to manage the server, view various information, resolve clogged operations and other purposes. These tools should only be used by developers or managers.

The MQ Platform uses a stripped-down GS application, since the MQ application does not require all the functionality of a highly loaded GS application. Many GS application tools are also required only for the BSG platform that implements slot games and various other functionalities. Therefore, the MQ Platform may have artifacts in the form of links to non-existent tools that have simply not been removed from the MQ Platform.

Some useful GS application tools that can be used when working with the MQ Platform:

WalletManager — tool that allows to display stuck (pending) wallet player's operation and saved states of games (**lasthands**), as well as remove them manually.

It is possible to search by both AccountId and Ext.Id of the player (indicating the sub-casino and the bank).

Link: *https://gs_domain/tools/walletsManager.jsp*

WalletInfo — tool to view information about a player's wallet operations and lasthands. Also allows to remove stuck wallet operations. Displays more advanced information than WalletManager and is intended for use by developers.

Link: *https://gs_domain/support/walletInfo.jsp*

Logviewer — tool that allows to view the error logs of wallet operations using one of the following search options:

- SessionId (SiD)
- Support Ticket ID
- Token
- External Id
- Game Session Id
- Round Id
- Transaction Id

The search is carried out only for the last 12 days from the moment of the request.

Link: *https://gs_domain/support/logviewer.do*

CacheViewer — tool that allows to view objects in the selected (from available in the tool) cache of the application's GS.

Link: *https://gs_domain/support/cacheviewer.do*

Bank/SubCasino Configuration — tool that allows to view all the sub-casinos and banks of the system, as well as change bank/game configurations and more.

Link: *https://gs_domain/support/cache/bank/common/subcasinoSelect.jsp*

Server Configuration — tool that allows to view and edit the current server settings. Includes a set of various properties and their values.

Link: *https://gs_domain/support/serverConfiguration.do*

The MQ application also has a set of tools that are mainly intended for managers and testers:

Emulator tools — emulator used by game designers to fine-tune the math, enemy's trajectories on the map, and their spawn points.

Link: *https://mq_domain/teststandConfig.jsp?gameId=[GAME_ID]&bankId=[BANK_ID]* — math setting. For this tool to work, there must be a game room in a particular game of a particular bank.

Link: *https://mq_domain/support/mapConfig.jsp* — game map configuration.

Link: *https://mq_domain/support/spawnConfig.jsp?gameId=[GAME_ID]&bankId=[BANK_ID]* — configuration of enemy spawn points.

ATS — tool for managing game bots designed to increase game activity and interest from players.

Link: *https://mq_domain/ats/listAts.jsp*

Teststand — tool that allows to set the necessary features that will be assigned to the player in the game. Required for game testing. To work, tester must specify the SiD (SessionId) of the current game session. Only for MaxQuest games.

Link: *http://mq_domain/teststand.jsp?sid=[SID]*

Integration

MQ platform has only one integration of its games - Max Quest Brand ("CANEX" integration).

Game integration includes both game launch processes and interaction between the MQ Platform and the EC side (MQB) to exchange information about players and their bets/winnings during the games, as well as other additional interactions.

The basic unit of various integration configurations is the bank (BankInfo). The bank contains all configuration settings, including endpoints, available games, possible currencies, and so on.

To integrate Max Quest Brand, a custom integration protocol is used – CANEX protocol. This custom protocol is based on the standard BSG integration protocol - Standard CW v4.0 and has almost no differences. However, since MQ games do not support Bonus modes, as well as some other features that are provided in Standard CW v4.0, this custom protocol uses only a part of requests.

This integration implies only 2 banks. Each bank has a single currency available. Both currencies are fake. Other currencies must not exist on these banks. Banks must have the following identifiers and currency codes, respectively:

- 6274 — MMC.
- 6275 — MQC.

To test the integration, special stubs have been created, which must be set on banks instead of the necessary endpoints:

- Balance: *http://gs_domain/config/stub/canex/balance*
- Wager (Bet/Result): *http://gs_domain/config/stub/canex/wager*
- Refund: *http://gs_domain/config/stub/canex/refund*
- Authenticate: *http://gs_domain/config/stub/canex/auth*

At the current stage, endpoints for staging and live systems have not been provided by the EC side.

All bank settings, including games, endpoints, etc., are identical (excluding currency).

In this integration, all games from the list specified in the «Games and Modes» section, with the exception of "Triple Cash or Crash", must be issued to these banks.

A more detailed description of the Standard CW v4.0 integration protocol is in the document «BSG CW v3.08.pdf».

More detailed description of CANEX custom integration protocol in document «MQB_Integration_protocol.pdf».

Launching games

According to the integration, games can be launched in Guest, Free or Real mode. These are the standard launching procedures described in the standard integration. However, in the case of Action games, this only applies to SP mode of games. Launch examples:

- **Guest Mode** (does not require player authorization; only for demonstration)

http://environment_domain/cwguestlogin.do?bankId=6234&gameId=859&lang=en

- **Real/Free Mode** (requires player authorization)

http://environment_domain/cwstartgamev2.do?token=simpleTokenOfUserOnECSide&bankId=6234&gameId=859&mode=REAL&lang=en

However, BTG mode and CAF have different actions to launch.

- **BTG game mode**

http://environment_domain/battlegroundstartgamev2.do?gameId=862&bankId=6234&mode=real&token=simpleToken&buyIn=2000&homeUrl=homeUrl

Difference from the usual launch of games:

1. Using the *battlegroundstartgamev2.d*o method as opposed to *cwstartgamev2.do* for SP.
2. Mandatory passing of the «buyIn» parameter in the request. Only values available in game configurations can be used.

More details about CAF including launching games in the specification «MQB Challenge a friend.pdf».

Server Side

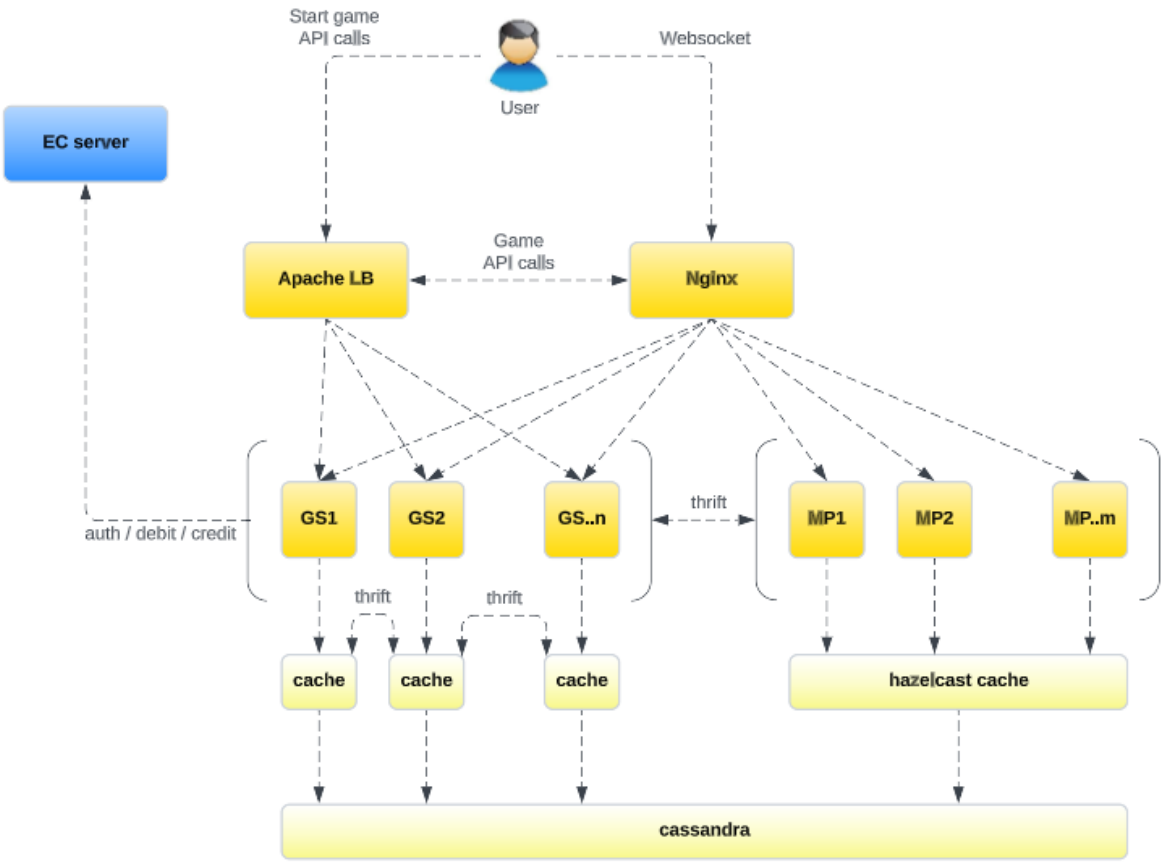
The server part of the MQ platform is represented by 2 Java applications:

- **Game Server (GS)** is the main application that implements such functionality as storing and processing player data, creating/editing/configuring and running integrations, interacting with the database, game launch processes, and more.
- **MQ (MP)** is an application that implements the server part of Action Games. Responsible for directly game logic (mathematics), the implementation of rooms (multiplayer), the gameplay of games, etc.

MQ application can only work together with GS application. Communication between applications takes place using Thrift.

The server part of the platform must be deployed on at least 2 instances of the cluster. Each instance must be running peer application instances. The minimum number of instances for a cluster is determined by the possibility of normal application updates without interrupting game activity, as well as greater fault tolerance.

The general architecture of the entire server part of the MQ platform is shown in the diagram below.



Based on this diagram:

- **Apache LB** — cluster load balancer. Entry point for all primary requests to the system (for example game start request). Later (during the game) Nginx is used.
- **Nginx** — distributes requests to the corresponding GS and MQ servers. In the event of a failure of the target server, it can redirect this request to another.
- **GS(1…N)** — GS Application.
- **MP(1…N)** — MQ Application.
- **Cassandra** — NoSQL distributed database. Used to store all existing information.
- **Cache (JVM-cache)** — there is one for each deployed GS. Used to store frequently accessed data. Reduces the number of slow operations between the GS and Cassandra. Thrift is used to ensure data consistency in the Caches of all GS's.
- **Hazelcast cache** — as well as «Cache (JVM-cache)» is used to cache data and speed up the receipt of this data by the application. It also provides data consistency between MP(1…N) applications without Thrift requests.

Communication with the game client occurs using **websockets**.

Communication with casino servers (EC servers) occurs using HTTP requests.

Both applications use the following technology stack (only basic technologies):

- Java 8 (Oracle Java 1.8)
- Maven 3.6
- Spring 5
- Hazelcast 3.12
- Cassandra 2.1
- Log4j2
- Apache Thrift

GS

The MQ platform uses a stripped-down GS application. The full version of the application implements many additional features necessary for the operation of the BSG platform that implements BSG slot games.

Some classes are allocated to separate module, because these modules are used in other projects of BSG (not only GSB); it relates for such modules as common-promo, kryo-validator, rng, sb-utils.

Project has next structure:

- /ansible — set of artefacts to deploy clusters on different systems (dev, staging (copy), stress testing, production) via TeamCity.
- /cassandra-cache — Cassandra persistence module.
- /cassandra-cache/cache — basic framework for work with Cassandra persistence module.
- /cassandra-cache/common-persisters — basic persisters of project:
    - to use persister it should be declared in config files (ClusterConfig.xml and SCClusterConfig.xml) at /game-server/web-gs/src/main/resources.
    - persisters that are required strong consistency are declared in SCClusterConfig.xml.
- /common — common classes of project.
- /common-wallet — support of common-wallet protocol and custom integrations.
- /deploy — secondary artefacts to deploy.
- /game-server — the main module of project.
- /game-server/build — build scripts for different environments.
- /game-server/common-gs — common classes of Game Server.
- /game-server/config — configuration files for different environments.
- /game-server/web-gs — Game server Web application, the core classes of project.
- /kryo-validator — mini framework for auto testing of Kryo serializing.
- /promo — module with promo framework.
- /promo/core — the core part of promo.
- /promo/persisters — persistence layer for promo.
- /rng — module with RNG.
- /sb-utils — module with common classes and utils for different projects
- /support/archiver — module for export and archive of data that are automatically removed from Cassandra (on TTL expiration).
- /utils — module with common classes and utils of project.

Basic entities

Cluster — group of servers and game resources that act like a single system and enable high availability, load balancing and parallel processing. Several GS and MP are associated with a cluster. Cluster entity has a relationship with the «Subcasino» entity.

Subcasino — entity that combines the main entities inherent in a particular licensee (EC system). This entity is associated with BankInfo, BaseGameInfo, BaseGameInfoTemplate.

BankInfo (Bank) — main entity that includes all the settings for a particular online casino that integrates MQ Platform games. This entity includes all integration settings, such as endpoints, games and their configurations, etc.

BaseGameInfo — entity describing all the games of the bank and their configurations.

AccountInfo — object containing all information about the player on the MQ Platform side. Main fields:

- id (accountId) — player ID on the MQ Platform side.
- externalId — player ID on the EC side (transferred to the MQ Platform side during the player's first authorization).
- bankId.

Combination bankId+externalId is unique.

SessionInfo — object that exists all the time from the player's authorization (the beginning of the game) to logout. Allows to identify the current game session of the player.

GameSession — entity designed to store data about a player's game session.

CommonWallet — object linking Account with a list of CommonGameWallet objects. Each CommonGameWallet object is associated with a player's gaming session, a specific round, and a specific payment transaction for that round.

MQ

The MQ (MP) project has the next structure:

- /ansible - ansible artfacts
- /bots - bot Server
- /build - build scripts for different systems
- /config - maven profiles for build
- /doc, /docs - documentation
- /core - common game classes implementation
- /core-interfaces - common game classes interfaces
- /games - concrete game implementations
- /persistance - persistance module (Cassandra)
- /tektosyne - BGS fork https://github.com/kynosarges/tektosyne
- /thrift-api - thrift API definition and classes for BSG GameServer
- /web - web module

Build and Deploy

Build

To build GS and MQ(MP) projects, need:

1. Check that Oracle Java 8 and Maven 3.6 are installed on the target machine.
2. Download project sources from GIT.
3. Execute the scripts specified in the readme-for_build-mq.txt of the GSN project (observing their sequence).

Deploy

At the moment, the MQ Platform is deployed on only one cluster – «mqb2».

GS1:

- 162.210.192.140:222
  gs1, gs1-test.maxquest.com
  gs., nt.

GS2:

- 207.244.72.56:222
  gs2, gs2-test.maxquest.com
  gs., nt.

GS3:

- 207.244.127.194:222
  gs3, gs3-test.maxquest.com
  gs., nt.

Cassandra:

- 1’st node: 162.210.192.175:224
- 2’d node: 199.115.115.153:224
- 3’d node: 23.105.164.250:224
- 4’th node: 23.105.164.250:224

Server for static content and NGINX:

- 108.62.117.49
  games-test.maxquest.com, games-mp-test.maxquest.com, lobby-test.maxquest.com, default-test.maxquest.com

For this cluster:

- Environment_domain for MQ TECNOLOGIA Subcasino (CANEX Integration): *default-test.maxquest.com*
- GS_Domain: *https://gs1-test.maxquest.com/*
- MQ_Domain: *https://gs1-mp-test.maxquest.com/*

At the moment, only manual issuance to this cluster is possible.

GS Issuance (for each GS on the cluster):

1. Execute the script «/game-server/build/live/build-mqb2-live.sh» for issuing GS in the required branch. The result is «ROOT.war»
2. Backup «/www/html/gs/ROOT/» if needed.
3. Check that the GS is running. Command «ps ax | grep jetty.gs»
4. Stop GS. Commands: «cd /www/restart», «gs.s»
5. Waiting for the GS to stop. Can check using «ps ax | grep jetty.gs»
6. After stopping GS, delete all the contents of the «/www/html/gs/ROOT/» directory.
7. Unzip «ROOT.war» into the user's temporary folder. For example in «/home/user1/upload/gs/»
8. Go to the folder from point 7 and execute the commands to change the rights:
   «find . -type f -name '*' -exec chmod u+rw-x,g+rw-x,o+r-wx \{\} \;»
   «find . -type d -name '*' -exec chmod u+rwx,g+rwxs,o+rx-w \{\} \;»
9. Move the entire contents of the folder from point 7 to «/www/html/gs/ROOT/» (with the exception of ROOT.war).
10. Start GS. Commands: «cd /www/restart», «gs.r»
11. Check the success of the start GS «ps ax | grep jetty.gs»
12. Go to «/www/logs/tomcat.gs» and analyze «common.log» for errors.

MQ(MP) Issuance (for each MP on the cluster):

1. Execute the script «/build/build-mqb2-live.sh» for issuing MP in the required branch. The result is «web-mp-casino.war»
2. Backup «/www/html/mp/ROOT/» if needed.
3. Check that the MP is running. Command «ps ax | grep Netty»
4. Stop MP. Commands: «cd /www/restart», «nt.s»
5. Waiting for the MP to stop. Can check using «ps ax | grep Netty»
6. After stopping MP, delete all the contents of the «/www/html/mp/ROOT/» directory.
7. Unzip «web-mp-casino.war» into the user's temporary folder. For example in «/home/user1/upload/mp/»
8. Go to the folder from point 7 and execute the commands to change the rights:
   «find . -type f -name '*' -exec chmod u+rw-x,g+rw-x,o+r-wx \{\} \;»
   «find . -type d -name '*' -exec chmod u+rwx,g+rwxs,o+rx-w \{\} \;»
9. Move the entire contents of the folder from point 7 to «/www/html/mp/ROOT/» (with the exception of web-mp-casino.war).
10. Start MP. Commands: «cd /www/restart», «nt.r»
11. Check the success of the start MP «ps ax | grep Netty»
12. Go to «/www/logs/tomcat.mp» and analyze «common.log» for errors.

Client Side

Frontend part of MQ/Crash games is designed on HTML5 technology and based on «Pixi.js» framework. Using «Node.js» it includes different modules to add some features and be able to build project into final code.

Basically, system have common part of game engine that contains «Pixi.js» framework and all general classes to work with game server, images, animation, sounds, translations, etc.

Every game was developed in separated catalogue, but export's common part during the build of project. The most games (expect Sector X) have all modes in one catalogue: SP, BTG, CAF. Meanwhile Sector X BTG was made separately from Sector X SP due to some external factors.

The following technology stack was used in the implementation:

- Javascript (ECMA 6)
- Node.js
- Babel.js
- Pixi.js (v.6)
- Vue.js
- Webpack